

Informe sobre patrones

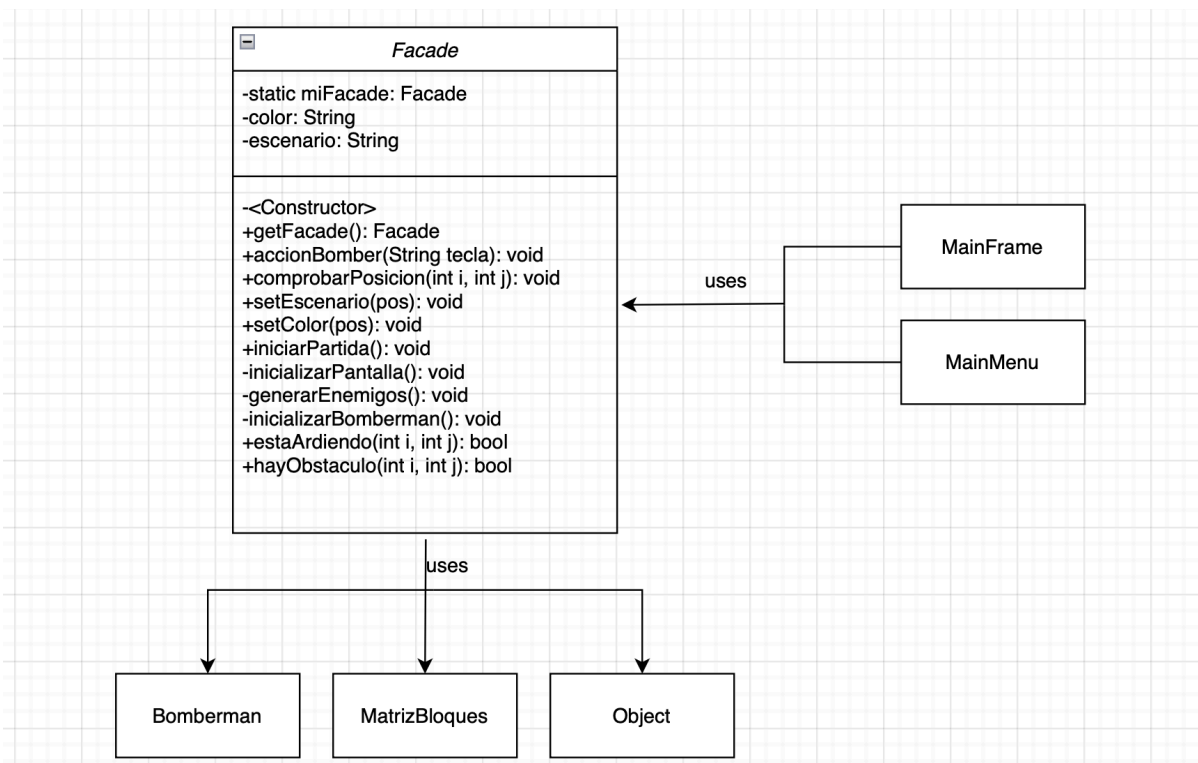
Facade

Simplifica un número de acciones separando las clases mediante un intermediario (clase **Facade**). Funciona como una interfaz para ocultar la complejidad de procesos como el de iniciar la partida, el que involucra iniciar el mapa, generar los enemigos y crear correctamente el **Bomberman**. Todos estos procesos ahora están dentro de un método llamado **iniciarPartida()** dentro de la fachada. Aquí podemos ver un resumen de sus métodos:

-accionBomber(tecla), comprobarPosicion(pos), setEscenario(pos) y setColor(pos) y iniciarPartida().

inicializarPantalla(), generarEnemigos() e inicializarBomberman() que son métodos privados llamados por **iniciarPartida()**.

-estaArdiendo(pos) y hayObstaculo(pos) devuelven información sobre dicha posición en la matriz.



Factory

Dado que es necesaria la creación de objetos como bloques y bombas de diferentes tipos, este patrón simplifica el proceso permitiendo que las clases puedan solicitar la creación de bombas y bloques sin conocer detalles.

La clase **Bomberman** solicita bombas de dos tipos (blancas y negras) mediante el método **generarBomba(tipo, coordenadas)**.

La clase **MatrizBloques** solicita la creación de bloques al comienzo de tres tipos (vacíos, blandos y duros) mediante el método **generarBloque(tipo)**.

El uso del patrón Factory permite aislar la lógica de creación de objetos del resto del sistema, facilitando futuras ampliaciones (por ejemplo, nuevos tipos de bombas o bloques) sin necesidad de modificar el código que los utiliza.

